



Lab 4. Setting Up Continuous Integration with Jenkins

By the end of this lab exercise, you should be able to:

- Set up Jenkins using Docker
- Take a walkthrough of the Jenkins web interface and essential configurations and start creating freestyle and maven jobs
- Integrate with tools such as Node.js and Maven
- Integrate with GitHub and set up build triggers
- Set up pipelines which run automated builds and unit tests

Install Docker Compose

Follow the directions at <https://docs.docker.com/compose/install/> to install Docker Compose on your machine.

Set Up Jenkins with Docker

You will learn how to set up Jenkins using Docker. Docker must be installed before this lab.

You will be running a Jenkins container on your Docker host by using the Jenkins image with version `jenkins/jenkins:2.492.3-lts-jdk21`. Use the following commands to clone the `devops-repo` directory and launch the Jenkins container:

```
git clone https://github.com/lftraining/devops-repo.git
```

```
cd devops-repo/setup
```

Inside the `devops-repo/setup` directory is the `docker-compose.yml` and the Dockerfile that we will be working with.

```
docker compose build
```

```
docker compose up -d
```

Access the Jenkins UI by browsing to `http://IPADDRESS:8080`. In this case, it will most likely be `http://localhost:8080` if you are on your own machine. If you are in the cloud, you will need to find the public IP of your cloud machine.

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

```
/var/jenkins_home/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password



Continue

To fetch the `initialAdminPassword` use the following command:

```
docker exec -it setup-docker-1 cat /var/jenkins_home/secrets/initialAdminPassword
```

The output will be the initial Jenkins password. Paste it into the Jenkins UI to unlock and click **Continue**.

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

```
/var/jenkins_home/secrets/initialAdminPassword
```

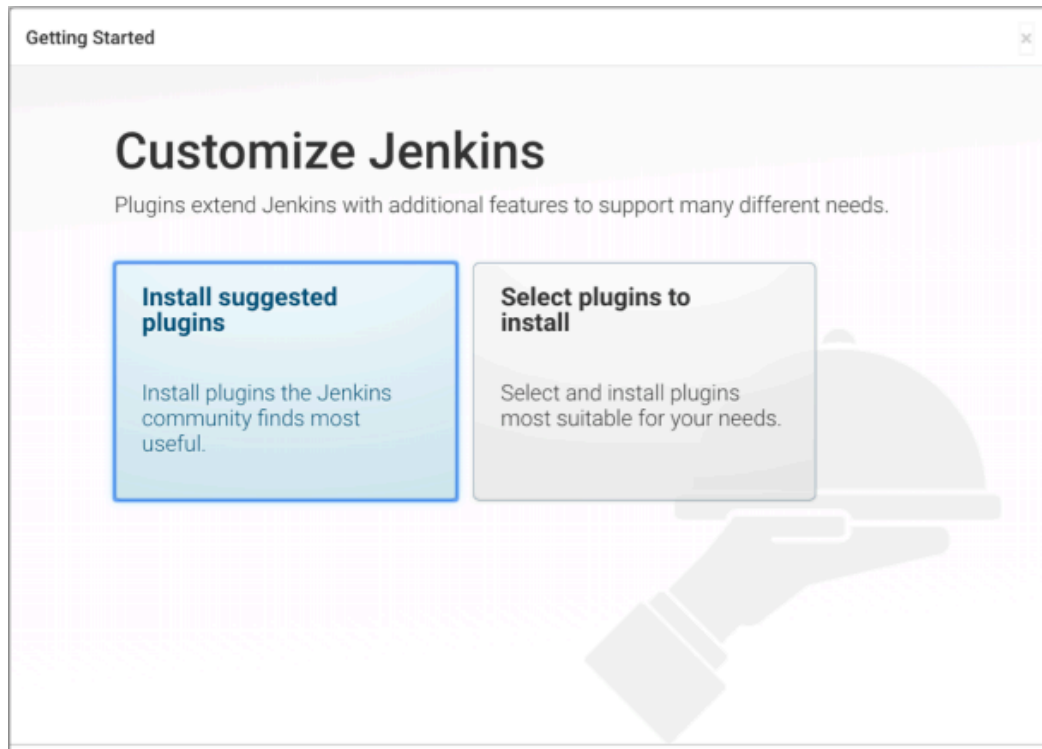
Please copy the password from either location and paste it below.

Administrator password



Continue

In the next step, choose **Install suggested plugins** to configure the default plugins automatically.



You will be able to observe the progress of the plugin installation process as follows:

Getting Started

Getting Started

✓ Folders Plugin	✓ OWASP Markup Formatter	✓ Build Timeout	✓ Credentials Binding Plugin	Folders OWASP Markup Formatter ** JSON Path API ** Token Macro Build Timeout Credentials Binding Timestamp ** Resource Disposer Workspace Cleanup Ant Gradle Pipeline ** GitHub GitHub Branch Source Pipeline: GitHub Groovy Libraries ** Metrics Pipeline Graph View
✓ Timestamp	✓ Workspace Cleanup	✓ Ant	✓ Gradle	
✓ Pipeline	✓ GitHub Branch Source	✓ Pipeline: GitHub Groovy Libraries	⌚ Pipeline Graph View	
⌚ Git plugin	○ SSH Build Agents	○ Matrix Authorization Strategy	○ PAM Authentication	
○ LDAP	⌚ Email Extension	○ Mailer Plugin	○ Dark Theme	
				** - required dependency

Once the plugins are installed, create the admin user using the form presented. Fill out the page and click **Save and Continue**. You can use a username and password of your choice.

Getting Started

Create First Admin User

Username

Password

Confirm password

Full name

E-mail address



On the **Instance Configuration** page, the URL will depend on whether you are on a local or cloud machine. The default should be fine for our purposes. Click **Save and Finish**.

Getting Started

Instance Configuration

Jenkins URL:

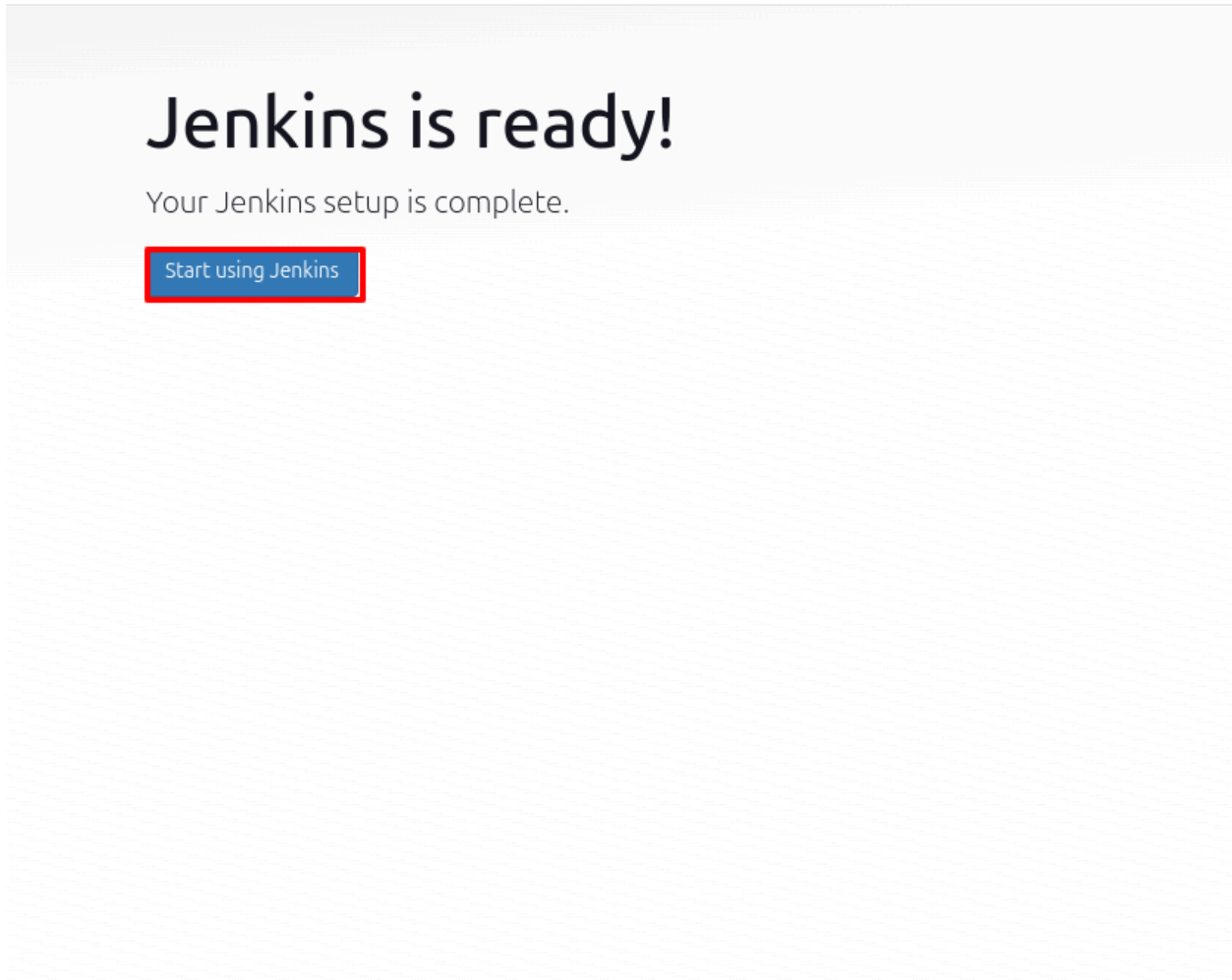
http://localhost:8080/

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

You should see a confirmation page. Click **Start using Jenkins** or **Restart**.

Getting Started



 **Jenkins**

Search (CTRL+K) ?

    admin   log out

Dashboard >

+ New Item

 Build History

 Manage Jenkins

 My Views

Build Queue

No builds in the queue.

Build Executor Status

1 Idle

2 Idle

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job +

Set up a distributed build

Set up an agent 

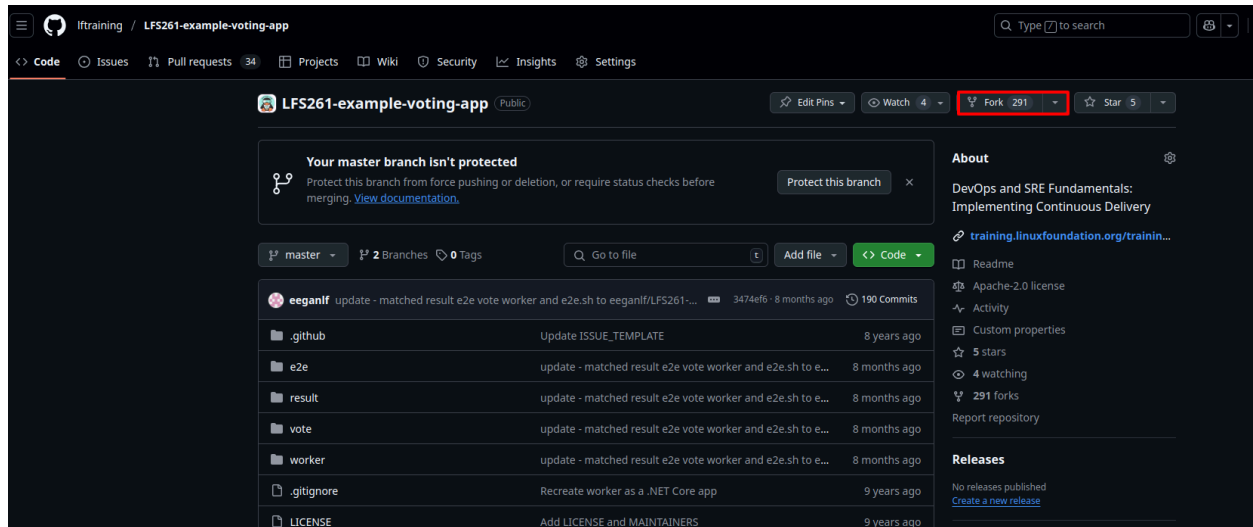
Configure a cloud 

Learn more about distributed builds ?

Add description

Fork the Voting App

Visit [example-voting-app on GitHub](#) and fork the repository onto your Git account.



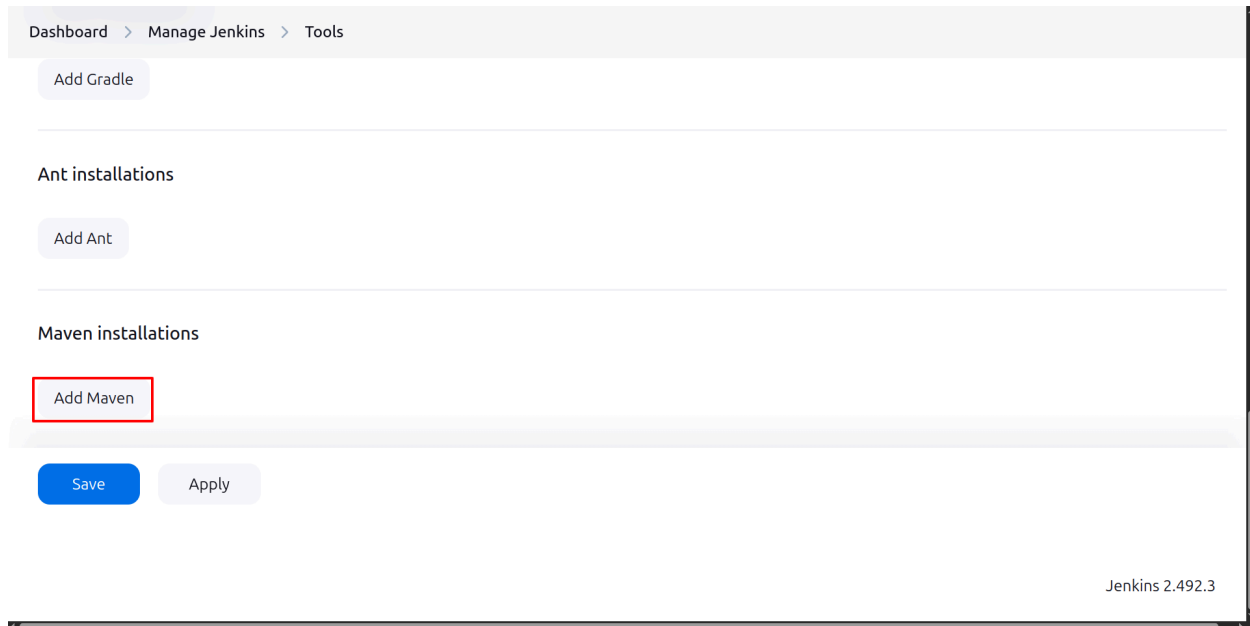
Clone the repository onto your own machine.

Configuring a Maven Build Job

We went over a simple `job-01` in Chapter 6. Now you will build the `worker` application as part of the `example-voting-app` project. This is a Java application that uses Maven as a build tool.

First we will configure a Maven build job.

Go to **Manage Jenkins > Tools** and, under the **Maven** section, click **Add Maven**.

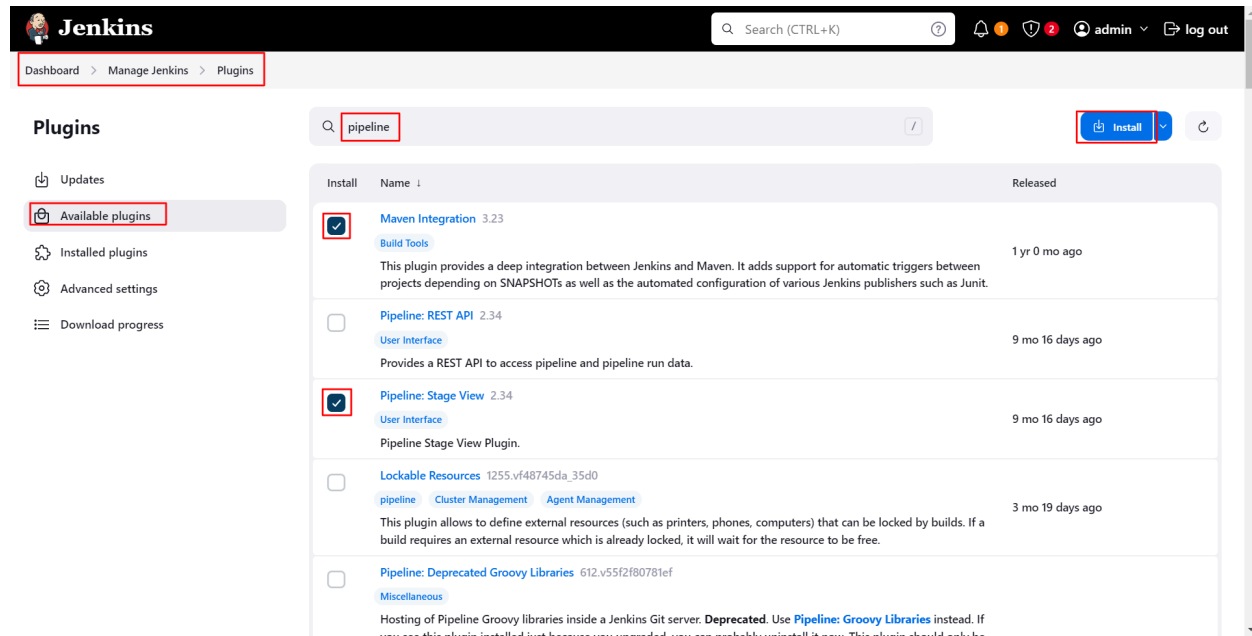


Paste `maven 3.9.8` into the name box and select maven version **3.9.8** from the **Version** dropdown menu under **Install from Apache**. Save the changes:

The screenshot shows the 'Add Maven' configuration interface in Jenkins. It includes a 'Name' field with the value 'maven 3.9.8', an 'Install automatically' checkbox that is checked, and a nested 'Install from Apache' section with a 'Version' dropdown set to '3.9.8'. At the bottom, there are 'Add Maven', 'Save', and 'Apply' buttons. The 'Save' button is highlighted with a red box.

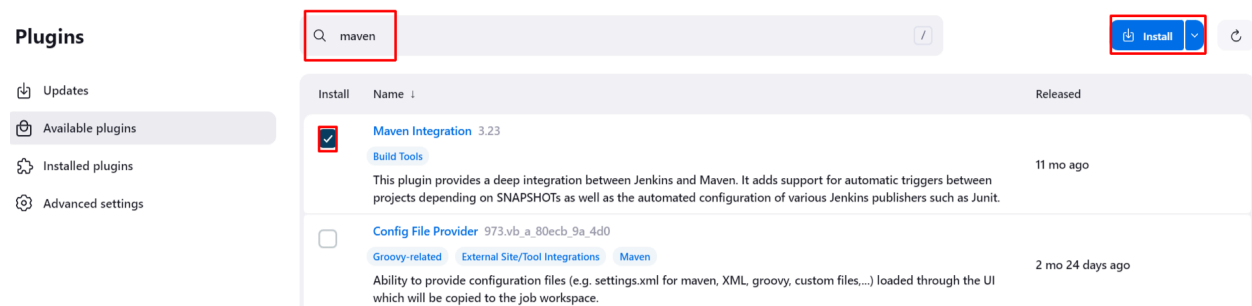
NOTE: Remember to use the exact name in this screenshot. This is the name you are going to use to reference the Maven installation in your Jenkinsfile later. If you provide a different name, be aware of it in future labs.

Install the Maven integration plugin. Go to **Manage Jenkins > Plugins > Available plugins**, search for **Maven Integration** and **Pipeline: Stage View** plugin and install them:



The screenshot shows the Jenkins web interface. The breadcrumb navigation at the top reads "Dashboard > Manage Jenkins > Plugins". On the left sidebar, the "Available plugins" option is selected. The main search bar contains the text "pipeline". Below the search bar, a table lists available plugins. The first two plugins, "Maven Integration 3.23" and "Pipeline: Stage View 2.34", have their checkboxes checked. The "install" button in the top right corner is highlighted with a red box.

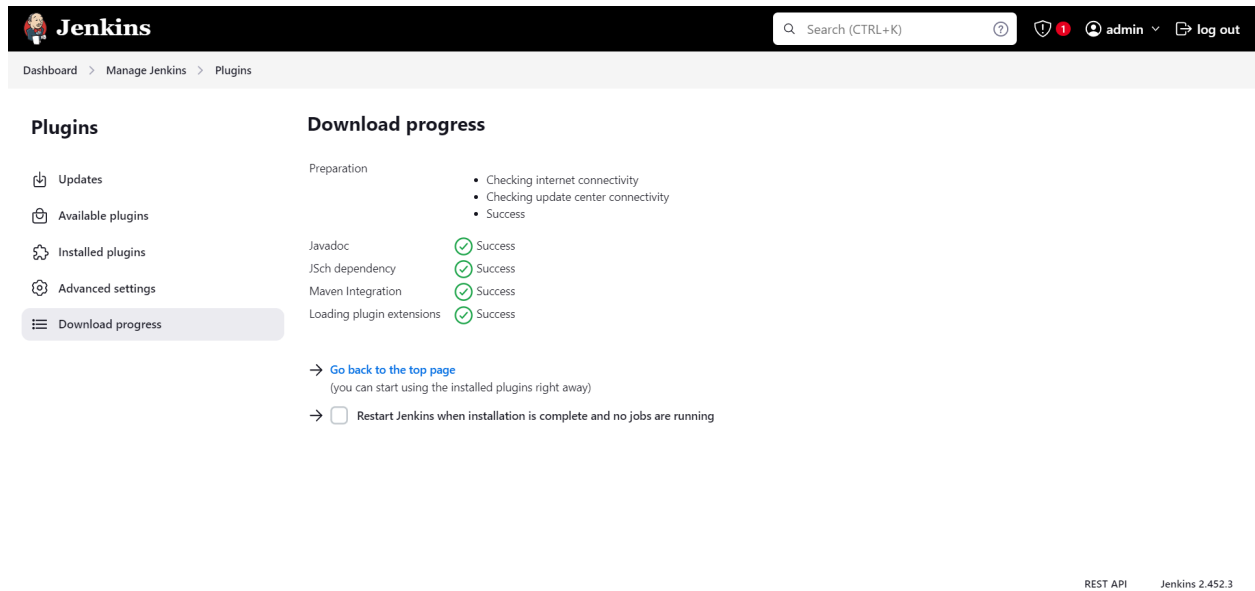
Install	Name	Released
☒	Maven Integration 3.23 Build Tools This plugin provides a deep integration between Jenkins and Maven. It adds support for automatic triggers between projects depending on SNAPSHOTS as well as the automated configuration of various Jenkins publishers such as Junit.	1 yr 0 mo ago
☐	Pipeline: REST API 2.34 User Interface Provides a REST API to access pipeline and pipeline run data.	9 mo 16 days ago
☒	Pipeline: Stage View 2.34 User Interface Pipeline Stage View Plugin.	9 mo 16 days ago
☐	Lockable Resources 1255.vf48745da_35d0 pipeline Cluster Management Agent Management This plugin allows to define external resources (such as printers, phones, computers) that can be locked by builds. If a build requires an external resource which is already locked, it will wait for the resource to be free.	3 mo 19 days ago
☐	Pipeline: Deprecated Groovy Libraries 612.v55f2f80781ef Miscellaneous Hosting of Pipeline Groovy libraries inside a Jenkins Git server. Deprecated. Use Pipeline: Groovy Libraries instead. If you see this plugin installed but haven't upgraded, you can probably uninstall it now. This plugin should not be	



The screenshot shows the Jenkins web interface with the search bar containing the text "maven". The table lists available plugins. The first plugin, "Maven Integration 3.23", has its checkbox checked. The "install" button in the top right corner is highlighted with a red box.

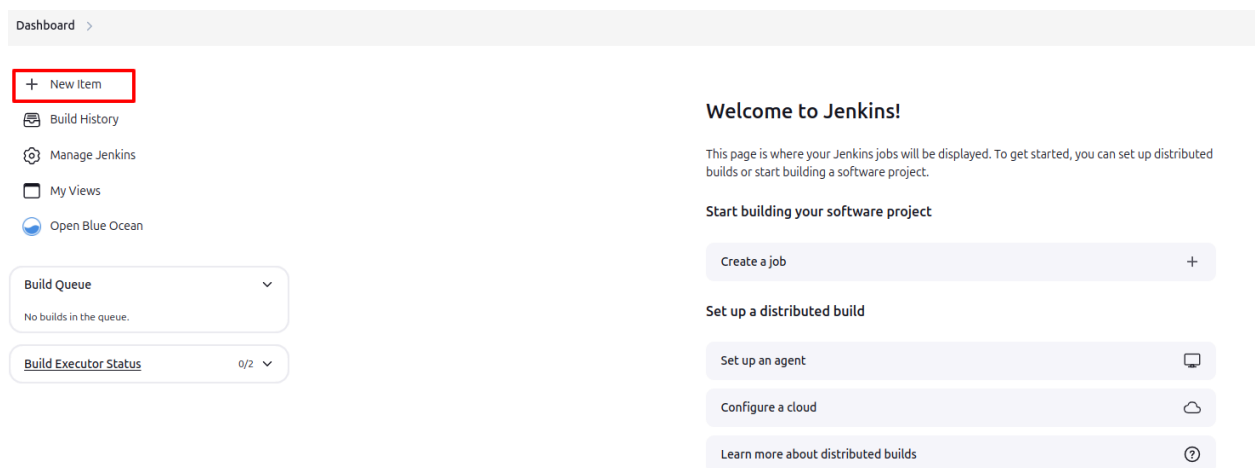
Install	Name	Released
☒	Maven Integration 3.23 Build Tools This plugin provides a deep integration between Jenkins and Maven. It adds support for automatic triggers between projects depending on SNAPSHOTS as well as the automated configuration of various Jenkins publishers such as Junit.	11 mo ago
☐	Config File Provider 973.vb_a_80ecb_9a_4d0 Groovy-related External Site/Tool Integrations Maven Ability to provide configuration files (e.g. settings.xml for maven, XML, groovy, custom files,...) loaded through the UI which will be copied to the job workspace.	2 mo 24 days ago

Click **Dashboard** in the top left of the page to go back to the home page.



The screenshot shows the Jenkins 'Plugins' page. The left sidebar contains a menu with 'Updates', 'Available plugins', 'Installed plugins', 'Advanced settings', and 'Download progress' (which is selected). The main content area is titled 'Download progress' and shows a list of plugins being installed: 'Preparation', 'Javadoc', 'JSch dependency', 'Maven Integration', and 'Loading plugin extensions'. Each plugin has a green checkmark and the word 'Success' next to it. Below the list, there are two links: 'Go back to the top page' and 'Restart Jenkins when installation is complete and no jobs are running'. The top navigation bar includes the Jenkins logo, a search bar, and user information (admin) and a 'log out' button. The bottom right corner shows 'REST API' and 'Jenkins 2.452.3'.

Create an **instavote** folder for your project. To do this, click **New item**:



The screenshot shows the Jenkins 'Dashboard' page. The left sidebar contains a menu with '+ New Item' (highlighted with a red box), 'Build History', 'Manage Jenkins', 'My Views', and 'Open Blue Ocean'. The main content area is titled 'Welcome to Jenkins!' and contains a 'Start building your software project' section with a 'Create a job' button. Below this is a 'Set up a distributed build' section with three buttons: 'Set up an agent', 'Configure a cloud', and 'Learn more about distributed builds'. The top navigation bar includes the Jenkins logo, a search bar, and user information (admin) and a 'log out' button. The bottom right corner shows 'REST API' and 'Jenkins 2.452.3'.

On the item creation page, enter the name as “instavote” and select the type as **Folder**. Click **OK**:

New Item

Enter an item name

Select an item type

**Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

**Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.

**Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

**Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

**Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

**Multibranch Pipeline**
Creates a set of Pipeline projects according to detected branches in one SCM repository.

**Organization Folder**
Creates a set of multibranch project subfolders by scanning for repositories.

On the configuration page that appears, fill out the required details as follows and **Save**:

Dashboard > instavote > Configuration

Configuration

- General
- Health metrics
- Properties

General

Display Name ?

instavote

Description

instavote folder

Plain text [Preview](#)

Health metrics

Health metrics ▼

Properties

Pipeline Libraries

Sharable libraries available to any Pipeline jobs inside this folder. These libraries will be untrusted, meaning their code runs in the Groovy sandbox.

Add

Save Apply

Inside the **instavote** folder, create a new item:

Jenkins

Dashboard > instavote >

- Status
- Configure
- New Item**
- Delete Folder
- Build History
- Favorite
- Open Blue Ocean
- Rename
- Credentials

instavote

instavote folder

All +

Add description

This folder is empty

Create a job +

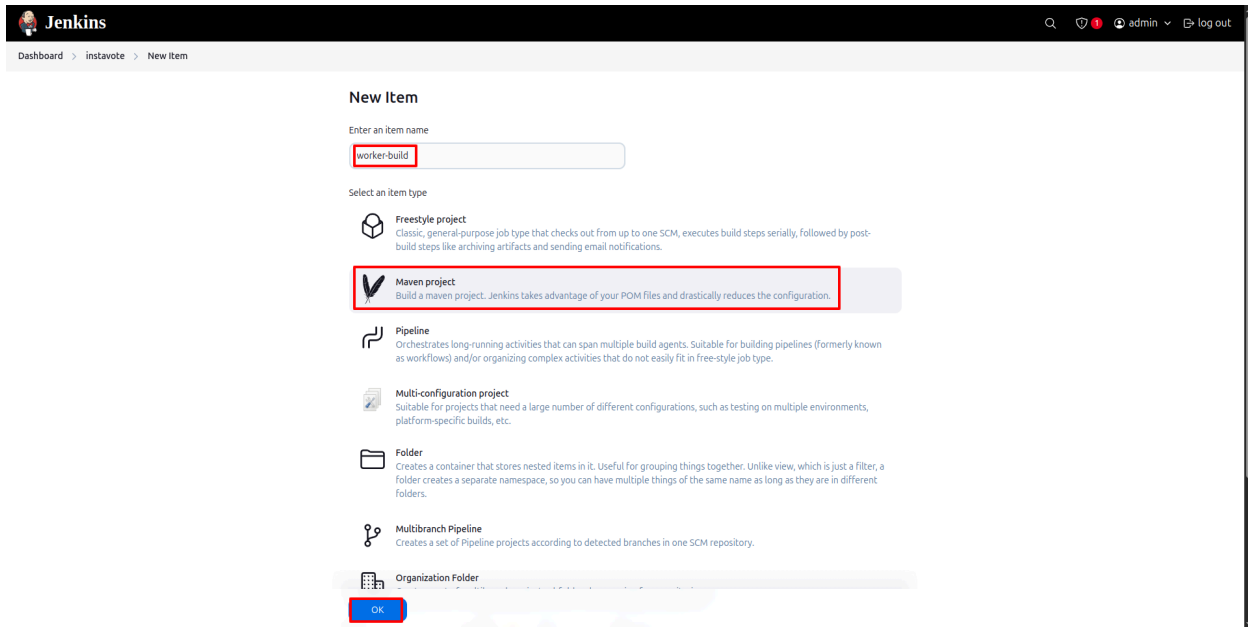
Build Queue ▼

No builds in the queue.

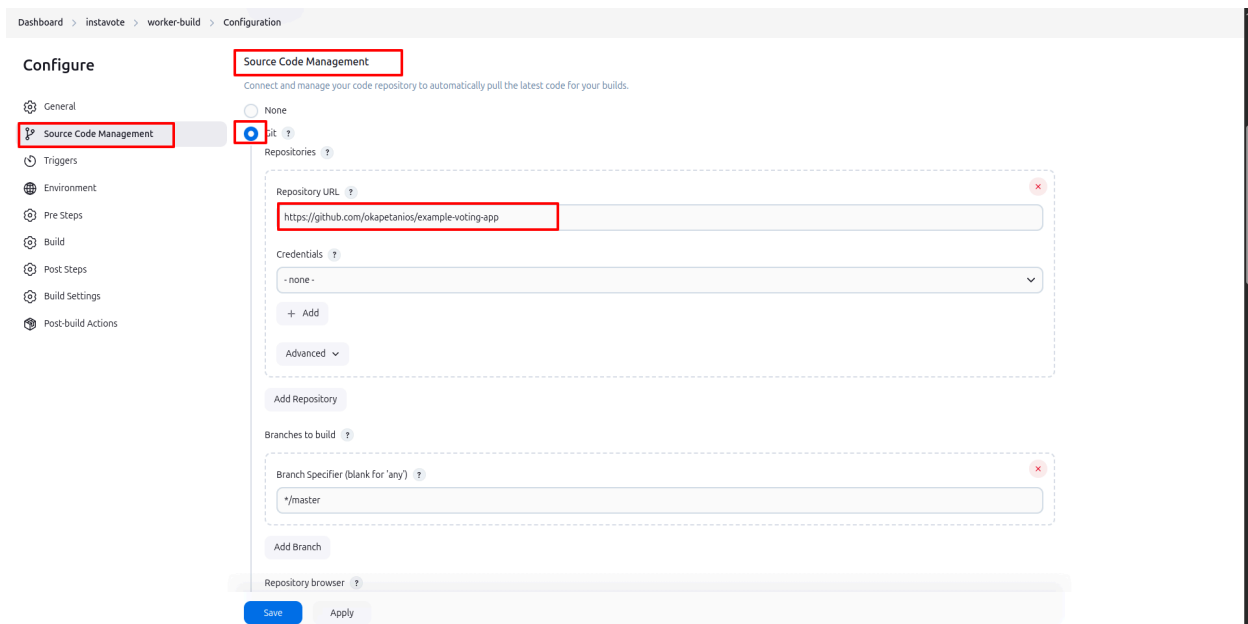
Build Executor Status 0/2 ▼

REST API Jenkins 2.492.3

Select **Maven project** as the project type. Name the job **worker-build** and click **OK**:



The **Configuration** page will appear. Scroll to the **Source Code Management** section and check the **Git** option. Provide the URL to your **example-voting-app** repository that you forked earlier.



Go to the **Build** section. Provide the path to `pom.xml`, which is in the `worker` subdirectory of the repo, i.e. `worker/pom.xml`. Type or paste **compile** into the **Goals and options** box.

Build

Root POM ?

worker/pom.xml

Goals and options ?

compile

Advanced ▼

Save the job and build.

Dashboard > instavote > worker-build >

 Status

 Changes

 Workspace

 Build Now

 Configure

 Delete Maven project

 Modules

 Move

 Favorite

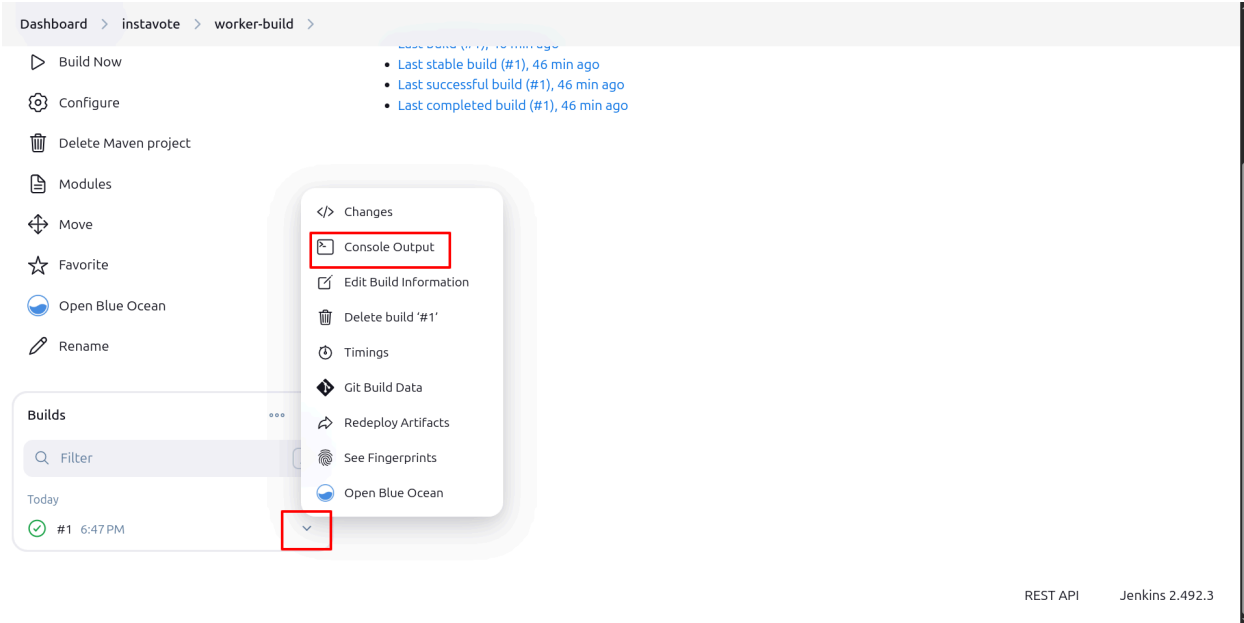
 Open Blue Ocean

 Rename

worker-build

Permalinks

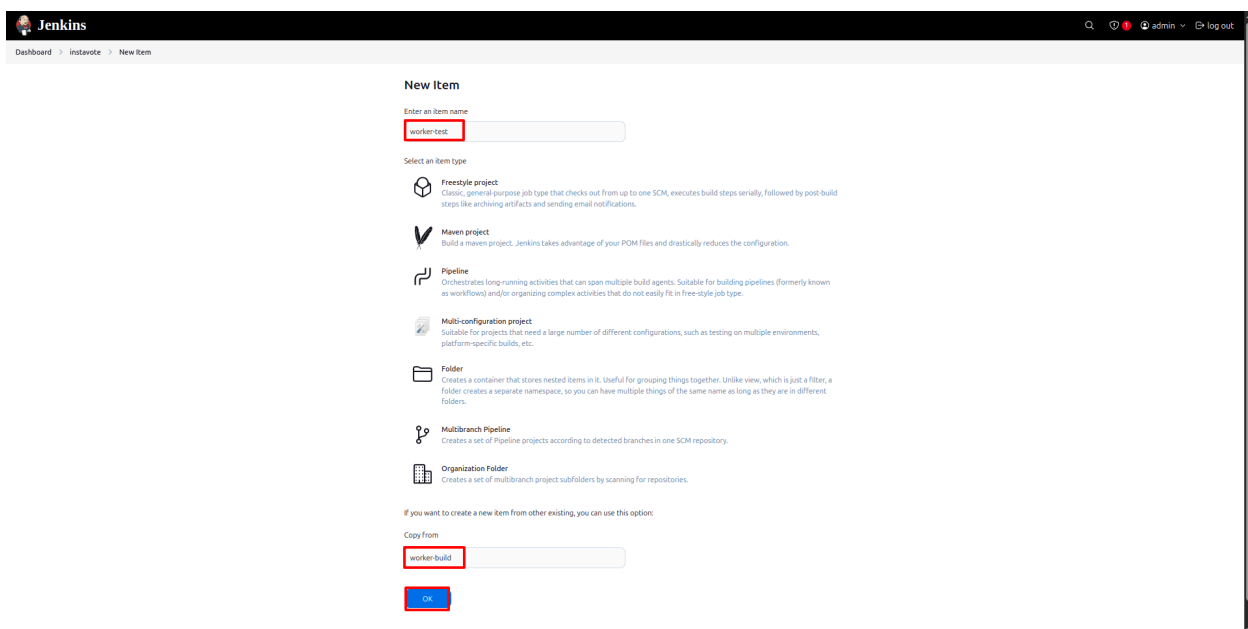
Observe the job status, console output, etc.



Adding Unit Test and Packaging Jobs

You will now add test and package jobs for the worker application.

You need to create one more job in the **instavote** folder and name it “worker-test”. You can copy the **worker-build** job.



Follow these steps to complete the configuration.

In the `worker-test` job, change your description to “test worker java app”. The source code management repository is the same.

Under the **Build** section, change the **Goals and options** field to “clean test” and leave the rest. Save the job and build.

Build

Root POM ?

worker/pom.xml

Goals and options ?

clean test

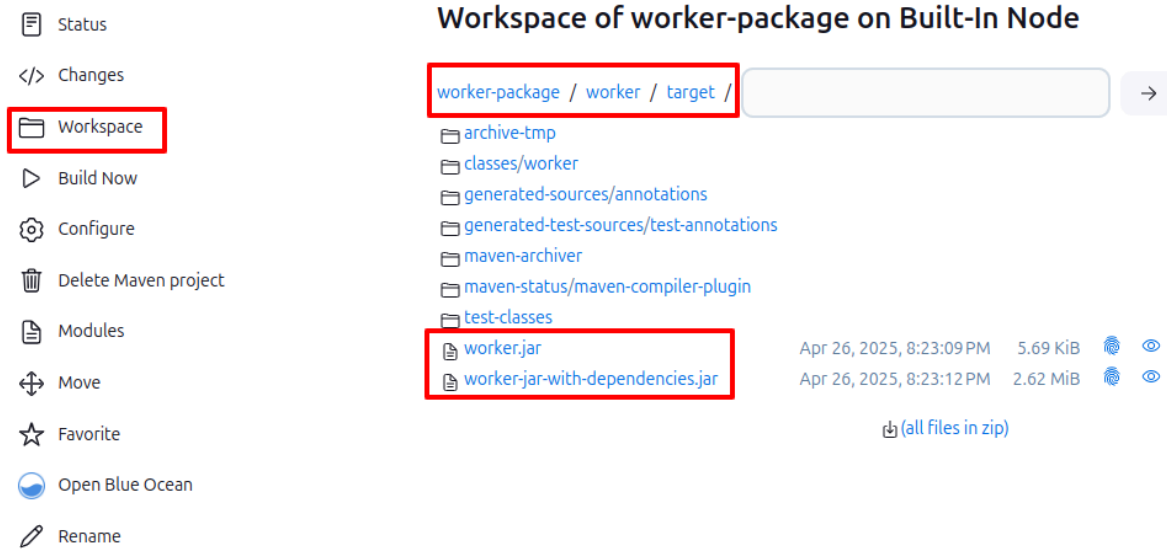
Advanced...

The next job will be `worker-package`. This will compile the application and then generate the `.jar` file. Create a job in the same folder named “worker-package”, copying the `worker-test` or the `worker-build` job.

Update the description to “package worker java app, create jar”. The only change in the configuration is in the Build step.

In the **Build** section for the package job, change the goal to “package -DskipTests”. Save the changes and build.

After the build is successful, you can see the `.jar` files created in the workspace under the `worker/target/` directory. You can verify your workspace by comparing it to the screenshot below.



Click the **Configure** option in the left panel.

From the post-build actions, choose **Archive the artifacts** and provide the path `**/target/*.jar` to store the `.jar` file.

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

≡ **Archive the artifacts** ?

Files to archive ?

`**/target/*.jar`

Advanced ▾

Add post-build action ▾

Save Apply

Save the changes and build the job.

Once the build is successful, check the project page to find your artifacts there.

Dashboard > instavote > worker-package >

Status

</> Changes

Workspace

Build Now

Configure

Delete Maven project

Modules

Move

Favorite

Open Blue Ocean

Rename

worker-package

package worker java app, create jar

Last Successful Artifacts

worker-jar-with-dependencies.jar 2.62 MiB view

worker.jar 5.69 KiB view

Permalinks

- Last build (#4), 7 min 5 sec ago
- Last stable build (#4), 7 min 5 sec ago
- Last successful build (#4), 7 min 5 sec ago
- Last completed build (#4), 7 min 5 sec ago

Configuring Build Triggers

You can use anything under **Build Triggers** to trigger automatic builds, but for now you are going to use **Poll SCM**.

Go to the `worker-build` job configuration page > **Triggers** and choose **Poll SCMs**. To periodically poll the Git repository, type or paste the following in the **Poll SCM** schedule:

```
H/2 * * * *
```


Once you have made changes, save the job, and go to your job page, where you will find the **Git polling log** in the left panel. Check your polling logs by clicking **Git Polling log**. It may take a minute to run.

Dashboard > instavote > worker-build > Git Polling Log

Status

Changes

Workspace

Build Now

Configure

Delete Maven project

Modules

Favorite

Git Polling Log

Move

Git Polling Log

```
Started on Nov 12, 2022, 3:40:00 PM
Using strategy: Default
[poll] Last Built Revision: Revision 4f256b896204526d787c804600380f192815f6cc
(refs/remotes/origin/master)
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git --version # timeout=10
> git --version # 'git version 2.30.2'
> git ls-remote -h -- https://github.com/eeganlf/LFS261-example-voting-app.git #
timeout=10
Found 2 remote heads on https://github.com/eeganlf/LFS261-example-voting-app.git
[poll] Latest remote head revision on refs/heads/master is:
4f256b896204526d787c804600380f192815f6cc - already built by 2
Done. Took 0.52 sec
No changes
```

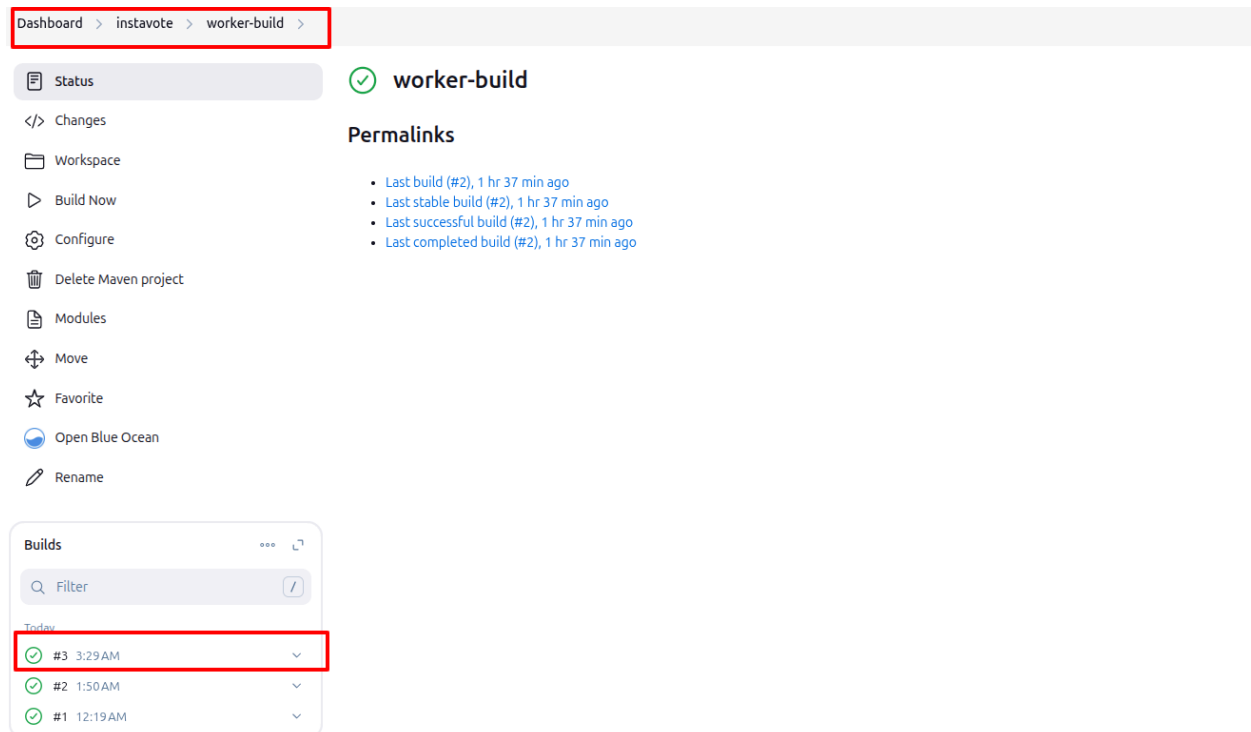
Now you will trigger builds remotely. Navigate back to **worker-build > Configuration** and scroll to the **Triggers** section. Select **Trigger builds remotely**. You can type any characters randomly into the token box, but be sure to remember what you put in as you will use it as the token to trigger a build.

A trigger URL is displayed just below where you defined the token. Your custom trigger URL will be similar to the following with your Jenkins instance IP address and port number.

TOKEN_NAME should be replaced with the text you entered into the **Authentication Token** text box:

```
http://IPADDRESS:8080/job/instavote/job/worker-build/build?token=TOKEN_NAME
```

Save the configuration. Enter your URL in the browser and press **Enter**; you *may* see a page asking you to proceed, but you may see no indication that anything has occurred. You can verify that the URL ran the build by checking **Build History** at the bottom of the left pane in your Maven **worker-build** project page and seeing that one more build has occurred:



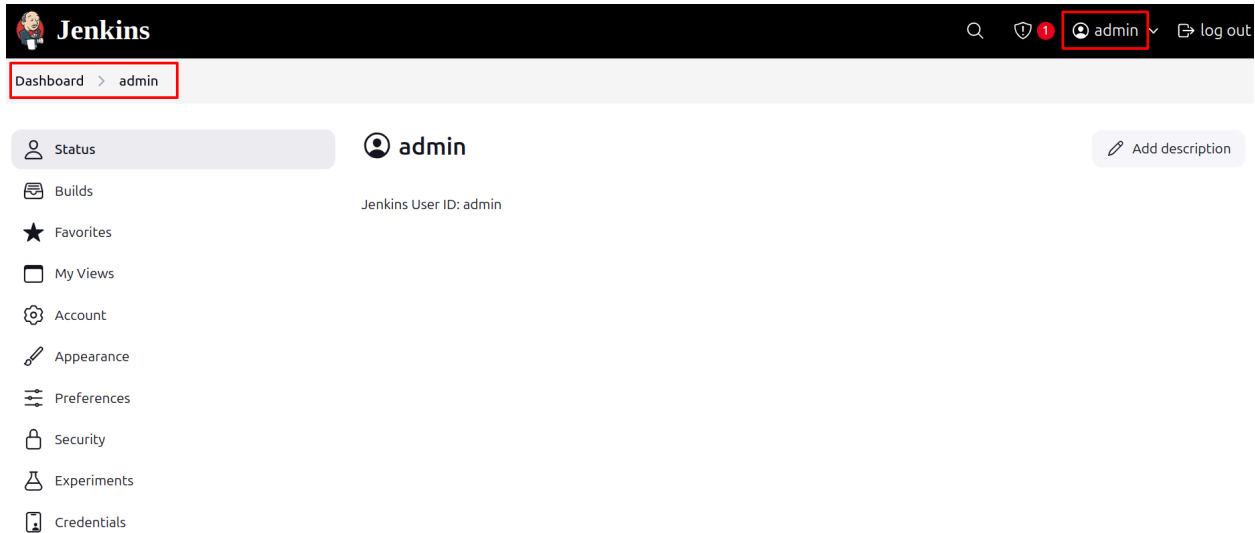
Click **Proceed** if the UI appeared and, since you are already authenticated, you should see the job launched automatically. Verify this from the project page.

You can't use this URL by itself outside of a browser session you are already logged into Jenkins with. You will need an API token tied to a Jenkins account for authentication and authorization.

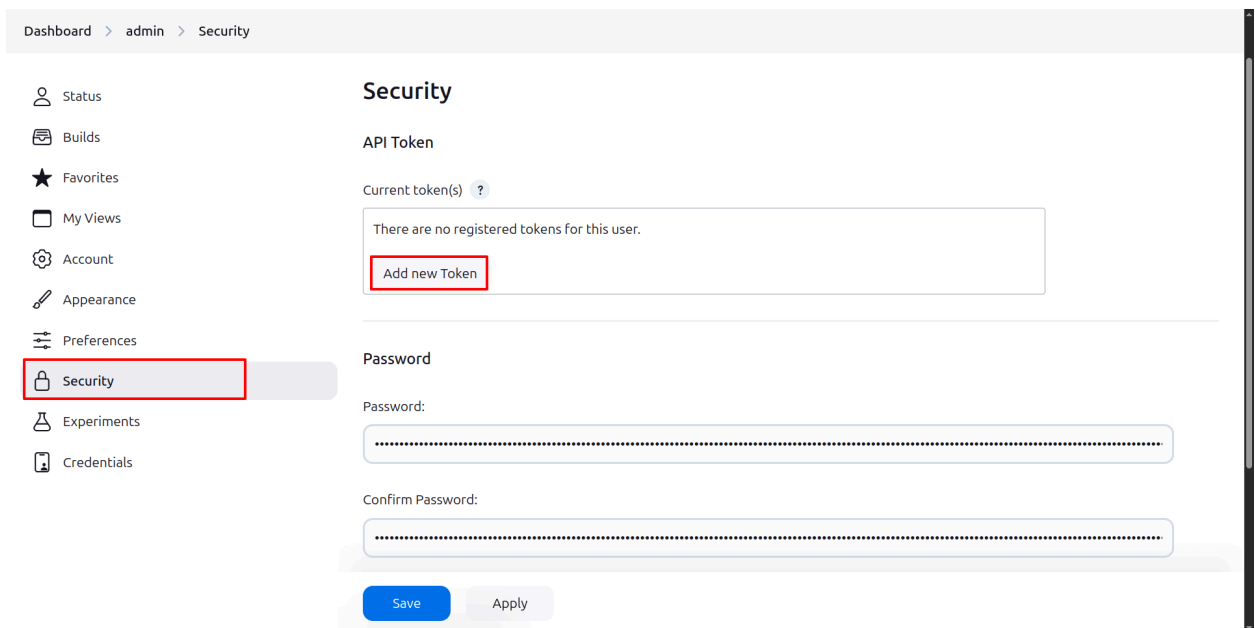
You can trigger the build using a command line interface or programmatically from external code

by providing an API token associated with a Jenkins user.

To create an API token, browse to **Jenkins -> UserAccount** as shown in the following image:



Browse to **Security** and **Add new Token**:



Name the token “cli-token” and click **Generate**:

API Token

Current token(s) ?

There are no registered tokens for this user.

cli-token **Generate**

Add new Token

API Token

Current token(s) ?

cli-token 11276bd2394ea3719da6d36b2e4dced78a  

⚠ Copy this token now, because it cannot be recovered in the future.

Add new Token

Copy the API token and put together a remote trigger URL similar to the following where **API_TOKEN** is the token you just created and **BUILD_TOKEN** is the token you entered in the **Triggers** step:

```
http://admin:API_TOKEN@IPADDRESS:8080/job/instavote/job/worker-build/build?token=BUILD_TOKEN
```

Test trigger the build using a `curl` command or equivalent:

```
curl
http://admin:API_TOKEN@IPADDRESS:8080/job/instavote/job/worker-build/build
```

`uild?token=BUILD_TOKEN`

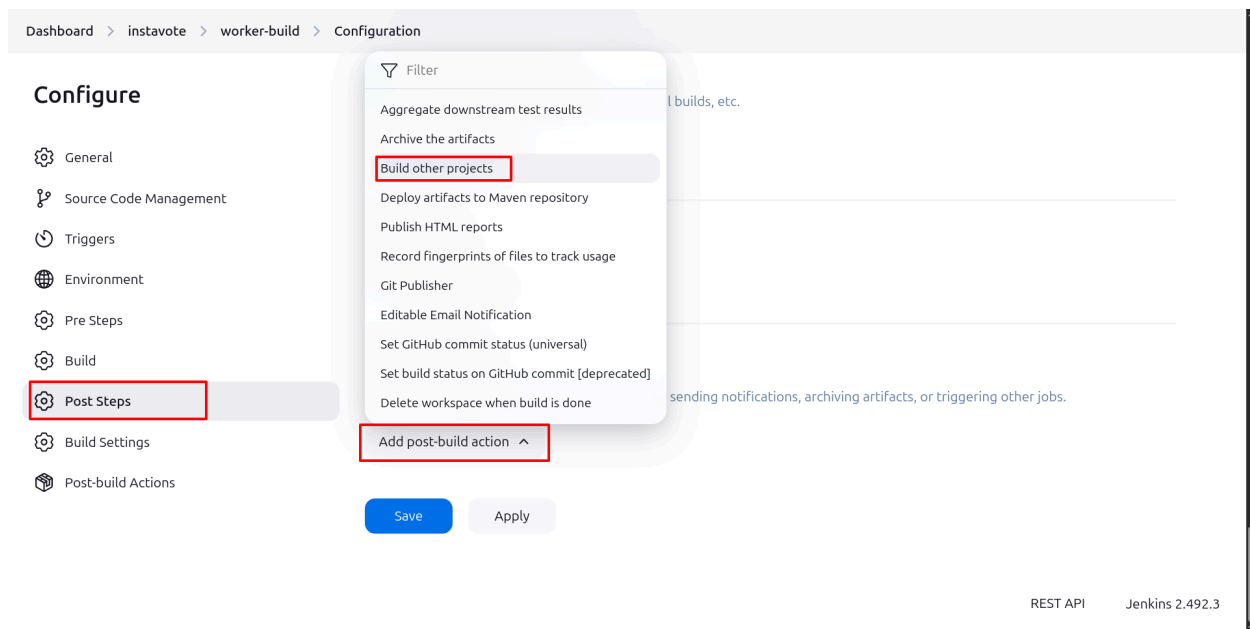
The above instructions demonstrate how to trigger builds remotely.

Creating a Job Pipeline

Now you will link jobs by defining upstreams and downstreams. You will also create a pipeline view using a plugin.

Follow these steps to set up upstream and downstream jobs.

From the `worker-build` job configuration page, scroll all the way to **Post-build Actions**, click **Add post-build action** and select **Build other projects**.



This is where you will define the downstream job. Provide `worker-test` as the project to build and save.

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

≡ Build other projects ?

Projects to build

worker-test,

☒ Trigger only if build is stable
☐ Trigger even if the build is unstable
☐ Trigger even if the build fails

Add post-build action ▾

Save Apply

Now you are going to set up the upstream for `worker-package`. Go to the `worker-package` configuration page. From **Triggers**, check the box for **Build after other projects are built**. Put `worker-test` in the **Build after other projects are built** input box.

Dashboard > instavote > worker-package > Configuration

Configure

- General
- Source Code Management
- Triggers**
- Environment
- Pre Steps
- Build
- Post Steps
- Build Settings
- Post-build Actions

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☒ Build whenever a SNAPSHOT dependency is built ?
- ☐ Schedule build when some upstream has no successful builds ?
- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☒ Build after other projects are built ?

Projects to watch

worker-test

- ☒ Trigger only if build is stable
- ☐ Trigger even if the build is unstable
- ☐ Trigger even if the build fails
- ☐ Always trigger, even if the build is aborted

- ☐ Build periodically ?
- ☐ GitHub hook trigger for GITScm polling ?

Save Apply

This defines the upstream for **worker-package**.

Once upstreams and downstreams are defined, run **worker-build** and it will automatically run **worker-test** and **worker-package**.

Set Up the Pipeline View

Next, you are going to set up a pipeline view for this build job.

Begin by installing the Build Pipeline plugin from the **Manage Jenkins > Manage Plugins** page. Type *build pipeline* in the search box, select the **Build Pipeline** plugin and click **Install without restart**.

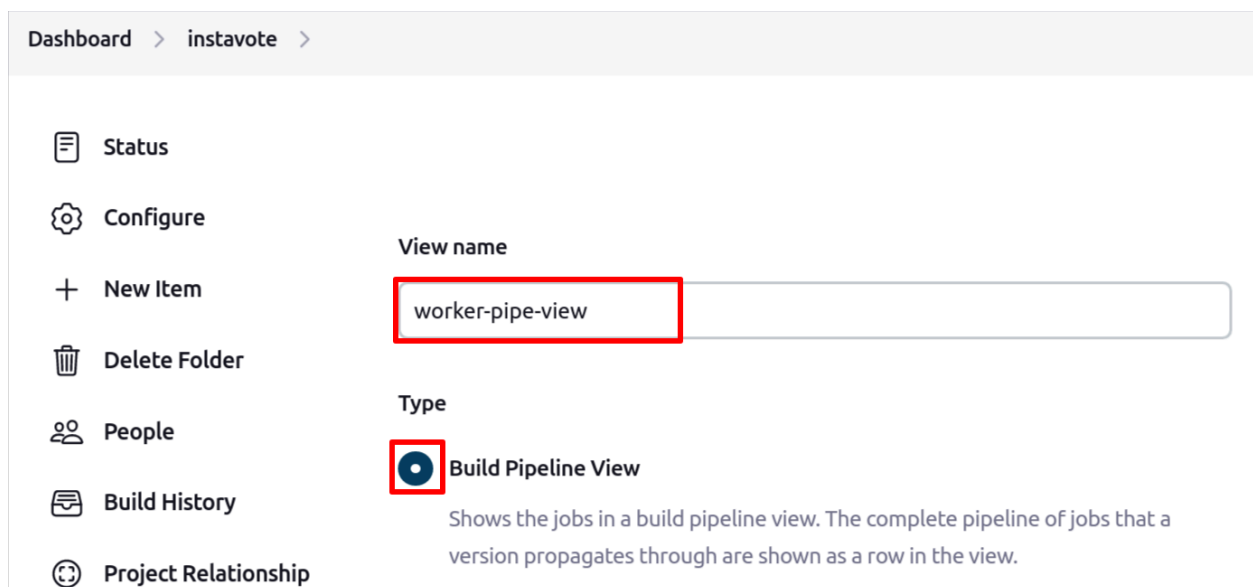
The screenshot shows the Jenkins 'Manage Plugins' interface. On the left sidebar, 'Available plugins' is selected and highlighted with a red box. The main area has a search bar containing 'build pipeline', also highlighted with a red box. Below the search bar, the 'Build Pipeline' plugin (version 1.5.8) is listed. It has a blue checkmark icon in a red box. The plugin description states: 'This plugin renders upstream and downstream connected jobs that typically form a build pipeline. In addition, it offers the ability to define manual triggers for jobs that require intervention prior to execution, e.g. an approval process outside of Jenkins.' A warning box is visible: 'Warning: This plugin version may not be safe to use. Please review the following security notices: • [Stored XSS vulnerability](#)'. At the bottom, the 'Install without restart' button is highlighted with a red box. Other buttons include 'Download now and install after restart' and 'Update information obtained: 1 day 18 hr ago'.

NOTE: You may see a vulnerability warning while installing the Build Pipeline plugin. It's not recommended that you install this plugin in a production environment. You are using this plugin only during this lab to aid your understanding of creating pipelines. Starting with the next chapter, you will use the Jenkinsfile, which does not need this plugin to provide you with a pipeline view. Jenkinsfiles are used in a real production environment.

Click on the **instavote** folder and you will now notice a **+** sign under the title. Click on it:



Select **Build Pipeline View** and provide a name for it, e.g. “worker-pipe-view”, then click **Create**:



From the configuration page, select the first job in the pipeline, **worker-build**, and select the number of displayed builds as 5, then save it.

Dashboard > instavote > worker-pipe-view > Edit View

Build Executor Status
(0 of 2 executors busy)

Based on upstream/downstream relationship

This layout mode derives the pipeline structure based on the upstream/downstream trigger relationship between jobs. This is the only out-of-the-box supported layout mode, but is open for extension.

Upstream / downstream config

Select Initial Job ?

instavote » worker-build

Trigger Options

Build Cards

Standard build card

Use the default build cards

Restrict triggers to most recent successful builds ?

☐ Yes

☒ No

Always allow manual trigger on pipeline steps ?

☐ Yes

☒ No

Display Options

No Of Displayed Builds ?

3

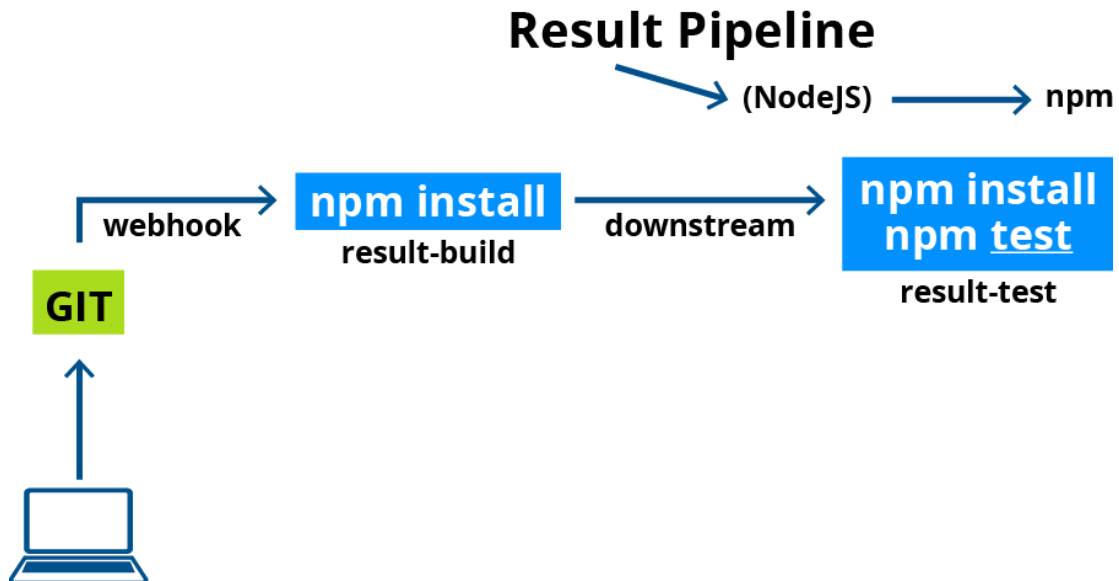
Save Apply

Once you complete, you will see the pipeline of your job. Green is successful, red is failed, blue is to do, yellow is in progress.

You have just created a pipeline for a Java project.

Exercise: Create a Pipeline for the Node.js Application

Now that you are familiar with creating CI pipelines, try creating one for the Node.js `result` application with `npm`.



Steps:

- Install **NodeJS** plugin for Jenkins.
- Configure **Global Tools** with a NodeJS installation with version 22.4.0.
- Create two jobs `result-build` and `result-test`, this time as freestyle projects.
- Define the Git repository you have forked in the **Source Code Management** section.
- Define the build trigger as **PollSCM** with an interval of 2 mins.
- In the build environment, choose to add NodeJS configurations with the version you have selected in **Global Tools**.
- Select the **Execute Shell** option from build and provide the commands to build the Node application from the `result` subdirectory. For example:

```
cd result
```

```
npm install
```

```
npm test
```

Summary

In this lab, we set up Jenkins using Docker Compose. We forked the `instavote` app so that we could work with our version of the app. We then used Jenkins to set up an integration pipeline for our `instavote` app on GitHub. Finally, we added a visual representation of the build pipeline.